



上海地铁 客流量统计系统

班 级： 计拔 大数据 计算机

学 号： 2350752 2351351 2352880

姓 名： 田思宇 肖逸恺 张誉翔

2026 年 1 月 4 日

第1节 需求分析

1.1 系统背景与目标

截至 2025 年 11 月，上海地铁已达 21 条线路、518 座车站，日均客流量超千万人次。随着上海地铁网络日益庞大，对其海量客流进行精细化管理和深度分析的需求日益迫切。然而，现有公开数据集常在时空分辨率与数据维度上存在局限。

Sun 等人发布的上海地铁数据集以其 10 分钟高分辨率、覆盖 302 个车站的进出与 OD 流量、以及独特的“通勤/家基/非家基”客流分类，有效弥补了这些缺口。基于此，我们构建了一个结构化的数据库，旨在将这一多维度数据集系统整合，为后续的客流预测、运营优化及城市研究提供坚实的数据基础。

1.2 性能需求

- 系统响应时间：**公众路径规划查询响应时间不超过 2 秒（95% 分位），高峰期并发用户数不低于 5000 时仍保持稳定。
- 数据查询性能：**历史客流多维度查询（包括时段、站点、OD、客流类型等）在亿级记录规模下，复杂聚合分析响应时间不超过 5 秒。
- 系统可用性：**年可用率 $\geq 99.9\%$ ，支持水平扩展，能够应对节假日客流高峰带来的突发流量冲击。
- 数据刷新：**实时估算行程时间每 5 分钟更新一次；动态客流仪表盘数据延迟不超过 10 分钟。

1.3 用户需求

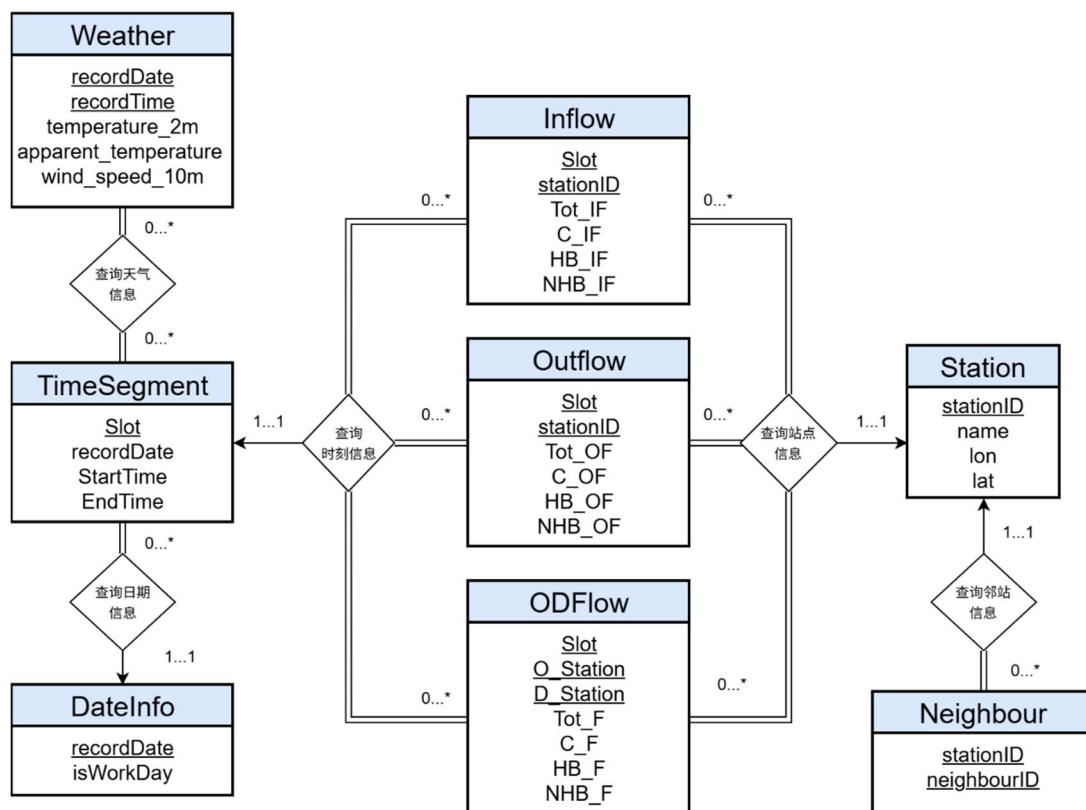
- 公众用户：**基于上海地铁网络拓扑和实时估算的行程时间，提供点对点的最优路径规划（最短时间、最少换乘、可步行距离最短等多种策略），并清晰展示详细换乘方案（包括换乘站、换乘时间、步行距离、出入口建议等）。
- 运营与规划人员：**支持对历史客流数据进行多维度深度挖掘，包括查询任意时段、任意站点/线路的进出站流量与 OD 流量；分析通勤（C）、家基（HB）、非家基（NHB）等不同类型的客流的时空分布规律，为列车时刻表优化、营销策略制定、容量扩充以及应急

管理提供数据驱动的决策支持。

1.4 功能需求

- **用户管理与互动：**支持用户注册、登录（支持手机号、微信等多种方式）；提供个人出行历史记录查询与统计功能；建立基于站点的留言板系统，允许用户发布和查看实时信息（如“高峰期某站台拥挤度”“某线路延误情况”等），并支持举报与审核机制，以增强系统社区互动性。
- **客流分析与可视化仪表盘：**集成动态客流仪表盘，支持实时与历史模式切换；以热力图展示全网或指定区域的客流密度分布；以趋势图、柱状图等形式展示指定站点/线路的进出站流量及 OD 客流随时间变化规律.....

第2节 概念设计 (ER 图)



第3节 实体设计

本节对数据库中的核心实体进行详细设计与说明。各实体表的设计旨在存储上海地铁客流数据的多维度信息，支持高效的时空查询、流量聚合以及后续的分析与可视化应用。实体设计遵循关系型数据库规范，确保数据完整性、一致性和查询效率。

3.1 站点信息 (Station)

站点信息表 (Station) 用于存储上海地铁各车站的基本属性，是整个数据库的空间基础实体，支持路径规划、客流统计及地图可视化等功能。

- 车站编号 (stationID)** 作为主键和唯一标识，便于与其他流量表建立外键关联，确保数据快速定位和追溯。
- 车站名称 (name)** 记录中文站名，用于用户友好显示和按名称模糊查询，便于公众路径规划中的站点搜索。
- 地理坐标 (lon, lat)** 以十进制经纬度形式存储车站位置，支持空间分析、热力图绘制以及与外部地理信息的整合（如计算步行距离或出入口建议）。

表名: Station			
字段	类型	约束	说明
stationID	INT	主键	站点唯一标识
name	VARCHAR(100)	非空	站点名称
lon	DECIMAL(11,8),		站点经度\°
lat	DECIMAL(11,8),		站点纬度\°

3.2 邻站关系 (Neighbour)

邻站关系表 (Neighbour) 用于描述地铁网络的拓扑结构，表示相邻车站之间的直接连接关系，是路径规划算法（如 Dijkstra 或 A*）的核心依据。

- 车站编号（stationID）和邻站编号（neighbourID）共同组成复合主键，确保每条邻站关系唯一且无向（若需区分方向，可在应用层处理）。
- 双外键约束分别引用 Station 表的 stationID，保证拓扑数据的完整性，避免无效或孤立车站引用，支持高效的图遍历查询。

表名: Neighbour			
字段	类型	约束	说明
stationID	INT	主键，外键	关联 Station 表的 stationID
neighbourID	INT	主键，外键	关联 Station 表的 stationID

3.3 日期信息（DateInfo）

日期信息表（DateInfo）用于标记每一天的属性，主要服务于客流时空分布规律分析。

- 记录日期（recordDate）作为主键，采用 DATE 类型，确保唯一性并便于范围查询。
- 是否工作日（isWorkDay）以 TINYINT(1)存储（1 表示工作日，0 表示周末/节假日），支持按工作日与非工作日对比分析客流差异（如通勤潮与休闲出行规律）。

表名: DateInfo			
字段	类型	约束	说明
recordDate	DATE	主键	日期
isWorkday	TINYINT(1)	非空	是否为工作日

3.4 天气逐小时信息（Weather）

天气逐小时信息表（Weather）存储外部气象数据，用于后续与客流数据的关联分析，探究天气因素对出行行为的影响。

- 主键由记录日期（recordDate）和记录时间（recordTime）复合组成，确保每小时天气

记录唯一。

- 包含温度（temperature_2m）、表观温度（apparent_temperature）、降雨量（rain）、风速（wind_speed_10m）等多项指标，支持多维度相关性分析（如雨天对进出站量的抑制效应）。
- 外键 recordDate 引用 DateInfo，支持按日期快速过滤。

表名: Weather			
字段	类型	约束	说明
recordDate	DATE	主键，外键	关联 DateInfo 表的 recordDate
recordTime	TIME	主键	时刻（每小时）
temperature_2m	DECIMAL(8,6)		距离地面 2 米温度值\°C
apparent_temperature	DECIMAL(8,6)		体感温度\°C
rain	DECIMAL(4,1)		降雨量\mm/h
wind_speed_10m	DECIMAL(10,6)		距离地面 10 米风速\km/h

3.5 时间段（TimeSegment）

时间段表（TimeSegment）对每日运营时间进行精细划分，是所有流量表的时间基准。

- 时段编号（Slot）作为主键，便于流量表的外键关联。
- 记录日期（recordDate）、起始时间（StartTime）、结束时间（EndTime）共同定义具体时间段，支持高分辨率（10 分钟级）客流聚合。
- 外键 recordDate 引用 DateInfo，确保时间段与日期属性一致。

表名: TimeSegment			
字段	类型	约束	说明

Slot	INT	主键	时间段编号
recordDate	DATE	非空，外键	关联 DateInfo 表的 recordDate
StartTime	TIME	非空	开始时间
EndTime	TIME	非空	结束时间

3.6 进站流量（Inflow）

进站流量表（Inflow）记录各车站、各时间段的进站客流量，是客流统计与可视化的核心数据源。

- 主键由**时段编号（Slot）**和**车站编号（stationID）**复合组成，保证每条记录唯一。
- 包含**总进站量（Tot_IF）**以及细分类型：**通勤类（C_IF）**、**家基类（HB_IF）**、**非家基类（NHB_IF）**，支持多维度客流类型分析。
- 双外键分别引用 TimeSegment(Slot)和 Station(stationID)，确保数据一致性并加速关联查询。

表名: TimeSegment			
字段	类型	约束	说明
Slot	INT	主键，外键	关联 TimeSegment 表的 Slot
stationID	INT	主键，外键	关联 Station 表的 stationID
Tot_IF	INT		总进站流量
C_IF	INT		通勤进站流量
HB_IF	INT		居家进站流量
NHB_IF	INT		非居家进站流量

3.7 出站流量 (Outflow)

出站流量表 (Outflow) 与进站流量表结构对称, 用于记录出站客流情况。

- 主键与字段设计同 Inflow, 包含**总出站量 (Tot_OF)**及 **C_OF**、**HB_OF**、**NHB_OF** 等细分类型。
- 支持与 Inflow 联合分析车站净客流、断面负荷等运营指标。

表名: Outflow			
字段	类型	约束	说明
Slot	INT	主键, 外键	关联 TimeSegment 表的 Slot
stationID	INT	主键, 外键	关联 Station 表的 stationID
Tot_OF	INT		总出站流量
C_OF	INT		通勤出站流量
HB_OF	INT		居家出站流量
NHB_OF	INT		非居家出站流量

3.8 起终点站流量 (ODFlow)

起终点站流量表 (ODFlow) 存储各时间段内起点-终点车站对的客流量, 是 OD 矩阵分析与客流预测的关键表。

- 主键由**时段编号 (Slot)**、**起点站 (O_Station)**、**终点站 (D_Station)** 三字段复合组成, 确保 OD 记录唯一。
- 包含**总 OD 流量 (Tot_F)** 以及 **C_F**、**HB_F**、**NHB_F** 细分类型, 支持精细化的客流来源-去向分析。
- 双外键 O_Station 和 D_Station 引用 Station(stationID), 便于快速获取站名与位置信息。

表名: ODFlow			
字段	类型	约束	说明
Slot	INT	主键, 外键	关联 TimeSegment 表的 Slot
O_Station	INT	主键, 外键	关联 Station 表的 stationID (出发站)
D_Station	INT	主键, 外键	关联 Station 表的 stationID (到达站)
Tot_F	INT		总 OD 流量
C_F	INT		通勤 OD 流量
HB_F	INT		居家 OD 流量
NHB_F	INT		非居家 OD 流量

第4节 逻辑设计分析

本节基于概念设计阶段的 E-R 模型, 对数据库进行关系模式转换与逻辑设计分析。设计过程严格遵循关系数据库规范化理论, 确保达到第三范式 (3NF), 消除数据冗余、避免插入/删除/更新异常, 同时充分考虑上海地铁客流数据的时空多维度特性与实际查询场景。

4.1 进出站流量信息 (Inflow & Outflow)

进站流量表 (Inflow) 和出站流量表 (Outflow) 用于记录各车站、各时间段的客流进出情况, 是系统进行站点级客流统计、拥挤度评估与断面负荷分析的核心数据源。

Inflow 与 Outflow 采用完全对称的独立表设计, 而非合并为单一表。这种分离避免了在同一记录中使用 NULL 值表示无进/出站情况, 减少了存储浪费, 同时便于独立进行进站或出站排名查询, 符合 3NF 要求并提升了聚合操作的简洁性。

4.2 起终点流量信息 (ODFlow)

起终点站流量表 (ODFlow) 用于存储各时间段内起点-终点车站对之间的客流量，是构建全网 OD 矩阵、分析客流来源去向与预测的基础表。

ODFlow 采用三字段复合主键 (Slot, O_Station, D_Station) 而非引入额外代理主键的设计，原因是 OD 记录本身具有天然的业务唯一性，且数据规模巨大。该自然主键方式可显著降低索引存储开销，同时直接体现“时间段+起终点对”的业务语义，便于后续分区管理与并行聚合查询。

4.3 时间与日期维度 (TimeSegment & DateInfo)

时间段表 (TimeSegment) 和日期信息表 (DateInfo) 共同构成客流数据的时间维度框架。

将“是否工作日”属性单独抽取至 DateInfo 表，避免在 TimeSegment 及三大流量表中重复存储 isWorkDay 字段，彻底消除冗余。该设计确保工作日/周末这一常用分析维度只需集中维护一次，即可全局生效，支持高效的客流模式对比。

TimeSegment 使用业务生成的 Slot 作为主键而非自增序列，原因是 Slot 已具备全局唯一性并携带明确的时间语义，便于流量表直接引用，同时支持快速反查具体时间段的起止时间，提升时间范围查询的便利性。

4.4 辅助分析维度 (Weather)

天气表 (Weather) 作为外部环境因素维度，用于与客流数据进行关联挖掘。

Weather 采用 (recordDate, recordTime) 复合主键的逐小时粒度设计，而非强制与 TimeSegment 的 10 分钟运营粒度对齐。该设计尊重气象数据原始分辨率，避免人为插值引入误差，保持数据真实性与分析可靠性。

在实际应用中，可通过时间重叠匹配实现 Weather 与 TimeSegment 的柔性关联，例如统计不同降雨强度下对应时间段的客流变化幅度，从而量化天气对地铁出行选择的影响。

第5节 物理设计 (索引结构)

为提升数据库的查询性能，设计高效的索引结构至关重要。数据库系统会自动为各表的主键创建唯一索引，这不仅确保了记录的唯一性，还支持快速查找和关联查询。

在设计其他索引结构时，我们遵循以下原则：**普通索引**用于加速基于单一字段的过滤查询，能够快速定位数据；**复合索引**则优化涉及多个字段的组合条件查询，有效减少查询范围。以下是各关键索引的设计理由与用途分析。

5.1 Station 表

普通索引	
字段	说明
name	支持按站点名称快速查询。在日常业务中，用户或分析系统常需通过站点名称（如“人民广场”、“虹桥火车站”）检索站点信息，该索引可避免全表扫描，显著提升响应速度。

5.2 TimeSegment 表

普通索引	
字段	说明
recordDate	支持按日期筛选时间段。系统常需分析某特定日期（如节假日、工作日）内的客流时段分布，此索引可加速 WHERE recordDate = '2025-11-01' 类型的查询，同时提升与 DateInfo 表关联时的 JOIN 效率。

5.3 Inflow 表

普通索引	
字段	说明
stationID	支持按站点筛选进站流量。尽管主键为 (Slot, stationID)，但若查询条件仅包含 stationID（如“查看 stationID 为 101 全天的进站流量”），主键索引因最左前缀限制无法生效。该索引确保此类高频分析操作高效执行，并加速与

	Station 表的关联。
--	---------------

5.4 Outflow 表

普通索引	
字段	说明
stationID	与 Inflow 表对称，用于高效查询某站点的出站流量。在进行站点吞吐量分析、进出站比对等场景中，该索引可避免全表扫描，大幅提升分析效率。

5.5 ODFlow 表

普通索引	
字段	说明
O_Station	支持按 <u>出发站点</u> 筛选 OD 流量。例如，“查询从‘南京东路’出发的所有出行记录”，是路径分析与起点热度统计的基础操作。
D_Station	支持按 <u>到达站点</u> 筛选 OD 流量。例如，“查询到达‘上海南站’的所有客流”，用于终点吸引力评估与接驳分析。
复合索引	
(O_Station, D_Station)	优化点对点组合查询。在进行特定 OD 对分析（如“A 站到 B 站的客流变化趋势”）时，该复合索引可直接定位记录，避免回表或临时排序，显著优于分别使用两个单列索引。

第6节 系统设计

6.1 系统总体架构

本系统采用前后端分离的架构模式，这是当前主流架构选择。在该架构下，前端负责用户界面的展示与交互逻辑，后端专注于业务处理与数据管理，两者通过标准化的 RESTful API 进行通信。

系统整体分为三个层次。表现层(Presentation Layer)、业务逻辑层(Business Logic Layer)、数据持久层(Data Persistence Layer)。表现层由 Vue 3 前端应用构成，运行在用户浏览器中，负责页面渲染与用户交互；业务逻辑层由 FastAPI 后端服务承载，处理 API 请求、执行业务规则、协调数据访问；数据持久层由 MySQL 数据库提供，存储系统所有结构化数据。

在技术选型上，我们综合考虑了开发效率、性能表现、生态成熟度等因素，最终确定如下技术栈。

层次	技术选型	说明
前端	Vue 3 + Vite + Element Plus + ECharts	响应式 UI 框架，支持丰富的数据可视化
后端	Python FastAPI + SQLAlchemy	高性能异步 API 框架，ORM 简化数据库操作
数据库	MySQL 8.0	关系型数据库，支持复杂查询与事务

针对上海地铁客流量统计系统的业务特点，我们采用了双数据库架构，将业务数据与分析数据进行物理隔离。这种设计基于数据特性差异。业务数据（用户、收藏、反馈）更新频繁但数据量较小；分析数据（客流、OD 流量）数据量庞大但以查询为主，两类数据的访问模式截然不同；性能隔离。复杂的客流聚合分析查询可能消耗大量数据库资源，独立的分析库可避免影响业务系统的响应速度；运维灵活。分析库可独立进行备份、优化、扩容，不影响业务系统的正常运行。

两个数据库的职责划分如下。

数据库	名称	存储内容
主库	jiading_commute	用户信息、用户收藏、拥挤度反馈、出行时刻表、地铁线路与站点元数据
分析库	shanghai_metro	站点信息、邻站拓扑、日期信息、时间段、天气数据、进站流量、出站流量、OD 流量

数据库连接配置实现灵活管理，支持开发、测试、生产环境的快速切换。核心配置代码如下。

```
# backend/config.py
import os
# 数据库配置
DATABASE_CONFIG = {
    "host": os.getenv("DB_HOST", "localhost"),
    "port": int(os.getenv("DB_PORT", 3306)),
    "user": os.getenv("DB_USER", "root"),
    "password": os.getenv("DB_PASSWORD", "123456"),
    "database": os.getenv("DB_NAME", "jiading_commute")
}

# SQLAlchemy 数据库连接 URL（主库）
DATABASE_URL = os.getenv("DATABASE_URL") or \
    f"mysql+pymysql://{DATABASE_CONFIG['user']}:{DATABASE_CONFIG['password']}@" \
    f"{DATABASE_CONFIG['host']}:{DATABASE_CONFIG['port']}/{DATABASE_CONFIG['database']}?charset=utf8mb4"

# 第二个数据库（上海地铁分析数据库）
DATABASE_URL_METRO = os.getenv("DATABASE_URL_METRO") or \
    f"mysql+pymysql://{os.getenv('DB_METRO_USER', 'root')}:{os.getenv('DB_METRO_PASSWORD', '123456')}@" \
    f"{os.getenv('DB_METRO_HOST', 'localhost')}:{os.getenv('DB_METRO_PORT', 3306)}/{shanghai_metro?charset=utf8mb4"
```

6.2 后端应用设计

后端基于 FastAPI 框架构建。FastAPI 是一个现代化的 Python Web 框架，以高性能著

称,同时具备自动生成 API 文档、原生类型校验等特性,非常适合构建 RESTful API 服务。

后端应用的入口文件为 `main.py`,负责创建应用实例、配置跨域中间件、注册各功能模块的路由。核心代码如下。

```
# backend/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from .routers import commute, metro, favorite, feedback, auth, metro_metro, time
# 创建 FastAPI 应用实例
app = FastAPI(
    title="嘉定出行后端 API",
    description="为嘉定智能出行决策页面提供数据接口",
    version="1.0.0",
    docs_url="/docs",      # Swagger UI 文档
    redoc_url="/redoc"     # ReDoc 文档
)

# CORS 跨域配置 (允许前端跨域调用)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 注册各功能模块路由
app.include_router(auth.router, prefix="/api/auth", tags=["用户认证"])
app.include_router(metro_metro.router, prefix="/api/metro", tags=["上海地铁"])
app.include_router(time.router, prefix="/api/time", tags=["时间"])
app.include_router(commute.router, prefix="/api/commute", tags=["出行"])
app.include_router(favorite.router, prefix="/api/favorites", tags=["收藏"])
app.include_router(feedback.router, prefix="/api/feedback", tags=["反馈"])
```

由于前后端分离架构下前端与后端运行在不同端口,浏览器的同源策略会阻止跨域请求。通过配置 CORS 中间件,允许前端应用正常调用后端 API。

后端采用模块化路由设计,按功能将接口划分为独立模块,每个模块定义自己的 `APIRouter`,再统一注册到主应用。这种设计使代码结构清晰、职责分明,便于团队协作与后期维护。

模块	路径前缀	功能说明
auth	/api/auth	用户注册、登录、信息查询与更新
metro_metro	/api/metro	站点列表、进出站流量、天气、OD 流量查询
time	/api/time	日期信息、时间段的增删改查
commute	/api/commute	出行时刻表查询
favorite	/api/favorites	用户收藏的增删查
feedback	/api/feedback	拥挤度反馈提交与查询

6.3 数据库连接与会话管理

系统使用 SQLAlchemy 作为 ORM 框架，负责数据库连接管理与对象关系映射。SQLAlchemy 是 Python 生态中最成熟的 ORM 解决方案，支持多种数据库后端，提供灵活的查询构建器和完善的连接池管理。

根据 6.1 节的双数据库架构设计，系统需要同时连接主库（jiading_commute）和分析库（shanghai_metro）。通过创建两个独立的数据库引擎实现。

```
# backend/database.py
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from .config import DATABASE_URL, DATABASE_URL_METRO

# 主库引擎 (jiading_commute)
engine = create_engine(
    DATABASE_URL,
    pool_pre_ping=True,      # 每次使用前检测连接有效性
    pool_recycle=3600,      # 每小时回收连接，防止超时断开
)

# 分析库引擎 (shanghai_metro)
engine_metro = create_engine(
    DATABASE_URL_METRO,
```



```

pool_pre_ping=True,
pool_recycle=3600,
)
# 创建会话工厂
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
SessionLocalMetro = sessionmaker(autocommit=False, autoflush=False,
bind=engine_metro)
# ORM 模型基类
Base = declarative_base()
    
```

6.4 ORM 数据模型设计

系统使用 SQLAlchemy ORM 定义数据模型，将数据库表映射为 Python 类。ORM（Object-Relational Mapping）模式使开发者可以用面向对象的方式操作数据库，无需编写原生 SQL 语句，提升开发效率并降低 SQL 注入风险。

主库业务模型：

主库 `jiading_commute` 存储系统业务数据，包含以下核心模型。

模型类	对应表	功能说明
User	users	用户信息，包含用户名、密码（哈希）、邮箱、昵称、头像、角色等字段
UserFavorite	user_favorite	用户收藏记录，关联用户 ID 与收藏的站点名称
CongestionFeedback	congestion_feedback	拥挤度反馈，记录用户上报的线路拥挤情况
MetroLine	metro_line	地铁线路基础信息
MetroStation	metro_station	地铁站点基础信息
MetroLineStation	metro_line_station	线路-站点多对多关联表

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, autoincrement=True)
    username = Column(String(50), unique=True, nullable=False)
    password = Column(String(255), nullable=False)
    email = Column(String(100), unique=True, nullable=False)
    role = Column(Enum('user', 'admin'), default='user')
```

模型设计要点如下：

- **主键设计：**所有表使用自增整数作为主键，保证唯一性和索引效率。
- **唯一约束：**用户名、邮箱设置唯一约束，在数据库层面防止重复注册。
- **枚举类型：**用户角色（user/admin）、拥挤等级（empty/normal/crowded/packed）使用枚举类型约束取值范围，保证数据规范性。
- **多对多关系：**地铁线路与站点之间是多对多关系（一条线路包含多个站点，换乘站属于多条线路），通过中间表 `metro_line_station` 建立关联，并使用 SQLAlchemy 的 `relationship` 实现双向导航查询。
- **字段注释：**通过 `comment` 参数为字段添加说明，便于数据库文档生成和后期维护。

6.5 后端 API 接口设计

后端 API 按功能模块划分，每个模块定义独立的路由文件，通过 FastAPI 的 `APIRouter` 实现模块化管理。所有接口遵循 RESTful 设计规范，使用标准 HTTP 方法表达操作语义。

接口模块总览如下。

模块	路径前缀	主要接口
用户认证	/api/auth	注册 POST /register、登录 POST /login、获取用户信息 GET /me
地铁数据	/api/metro	站点列表 GET /stations、进站流量 GET /inflow、出站流量 GET /outflow、天气 GET /weather、热门 OD GET /top-od

模块	路径前缀	主要接口
时间管理	/api/time	日期信息 GET /dateinfo、时间段增删改查
收藏管理	/api/favorites	添加收藏 POST /、删除收藏 DELETE /{id}、查询收藏 GET /
反馈管理	/api/feedback	提交反馈 POST /、查询反馈 GET /

- **用户认证接口：**用户认证模块实现完整的注册登录流程。注册时系统会校验用户名和邮箱的唯一性，密码使用 SHA256 算法哈希后存储，避免明文泄露风险。登录时验证用户名密码，成功后返回用户基本信息供前端保存。
- **地铁数据查询接口：**地铁数据模块是系统的核心，提供多维度的客流数据查询能力。
- **站点列表：**返回所有站点的编号、名称、经纬度坐标，支持分页限制
- **进出站流量：**支持按时间段（Slot）、站点（stationID）灵活过滤，返回总流量及通勤/家基/非家基分类流量
- **天气数据：**按日期查询逐小时天气记录，包含温度、体感温度、降雨量、风速等指标
- **热门 OD 流量：**聚合指定日期的起终点流量数据，按总流量降序返回 TOPN 热门线路。

6.6 前端项目结构

前端基于 Vue 3 框架开发，采用组合式 API（Composition API）编写组件逻辑，使用 Vite 作为构建工具，Element Plus 作为 UI 组件库，ECharts 实现数据可视化。

技术选型说明如下。

Vue 3：新一代 Vue 框架，组合式 API 提升代码复用性，响应式系统性能更优

Vite：基于原生 ES 模块的构建工具，开发服务器启动极快，热更新响应迅速，显著提升开发体验

Element Plus：企业级 Vue 3 组件库，提供表单、表格、弹窗、导航等丰富组件，开箱即用

ECharts：百度开源的可视化图表库，支持折线图、柱状图、饼图、桑基图等多种图表类型，满足客流数据可视化需求

TailwindCSS: 原子化 CSS 框架, 通过组合工具类快速构建响应式界面

目录结构:

```
src/
├── api/                # API 接口封装层
│   ├── authApi.js     # 用户认证相关接口
│   ├── commuteApi.js  # 通勤数据接口
│   └── shanghaiMetroApi.js # 地铁数据接口 (站点、流量、天气等)
├── components/        # 公共组件
│   ├── ECharts.vue    # ECharts 图表封装组件
│   ├── FeedbackDialog.vue # 拥挤度反馈弹窗
│   └── ScheduleEditDialog.vue # 时刻表编辑弹窗
├── views/             # 页面视图
│   ├── Login.vue      # 登录/注册页面
│   ├── Dashboard.vue  # 客流分析仪表盘
│   ├── RoutePlanner.vue # 路线规划页面
│   └── Commute.vue     # 通勤推荐页面
├── router/            # 路由配置
│   └── index.js        # 路由定义与守卫
├── styles/            # 全局样式
├── App.vue            # 根组件
└── main.js            # 应用入口
```

6.7 前端路由与权限控制

前端使用 Vue Router 实现单页面应用 (SPA) 的路由管理。Vue Router 是 Vue.js 官方路由库, 支持嵌套路由、路由守卫、动态路由匹配等特性, 能够满足复杂应用的导航需求。

路由配置如下, 系统定义了四个主要页面路由, 分别对应登录、仪表盘、路线规划和通勤推荐功能。

```
// src/router/index.js

import { createRouter, createWebHistory } from 'vue-router'
import Login from '@/views/Login.vue'
import Dashboard from '@/views/Dashboard.vue'
import Commute from '@/views/Commute.vue'
import RoutePlanner from '@/views/RoutePlanner.vue'

const routes = [
```

```
{ path: '/login', name: 'Login', component: Login },
{ path: '/', redirect: '/dashboard' },
{ path: '/dashboard', name: 'Dashboard', component: Dashboard, meta:
{ requiresAuth: true } },
{ path: '/route-planner', name: 'RoutePlanner', component: RoutePlanner, meta:
{ requiresAuth: true } },
{ path: '/commute', name: 'Commute', component: Commute, meta: { requiresAuth:
true } }
]

const router = createRouter({
  history: createWebHistory(),
  routes
})
```

6.7.1 页面路由说明

路径	页面	权限要求	功能说明
/login	登录页	无	用户登录与注册入口
/dashboard	仪表盘	需登录	客流统计与可视化分析（管理员视角）
/route-planner	路线规划	需登录	起终点选择与换乘方案查询（公众视角）
/commute	通勤推荐	需登录	嘉定校区出行时刻表与推荐

6.7.2 路由守卫实现

为保护需要登录才能访问的页面，系统通过 `beforeEach` 全局前置守卫实现权限控制。每次路由跳转前，守卫会检查目标页面是否需要认证（`meta.requiresAuth`），并验证用户登录状态。

```
router.beforeEach((to, from, next) => {
  const isAuthenticated = localStorage.getItem('isAuthenticated') === 'true'

  if (to.meta.requiresAuth && !isAuthenticated) {
    next('/login') // 未登录，重定向到登录页
  } else if (to.path === '/login' && isAuthenticated) {
```

```
next('/commute') // 已登录用户访问登录页，跳转到主页
} else {
  next() // 正常放行
}
})
```

6.7.3 登录状态持久化

用户登录成功后，前端将登录状态和用户信息存储到浏览器的 `localStorage` 中，实现页面刷新后状态保持

存储键	说明
<code>isAuthenticated</code>	登录状态标识 ('true'/'false')
<code>userId</code>	当前登录用户 ID
<code>username</code>	用户名
<code>userRole</code>	用户角色 (user/admin)

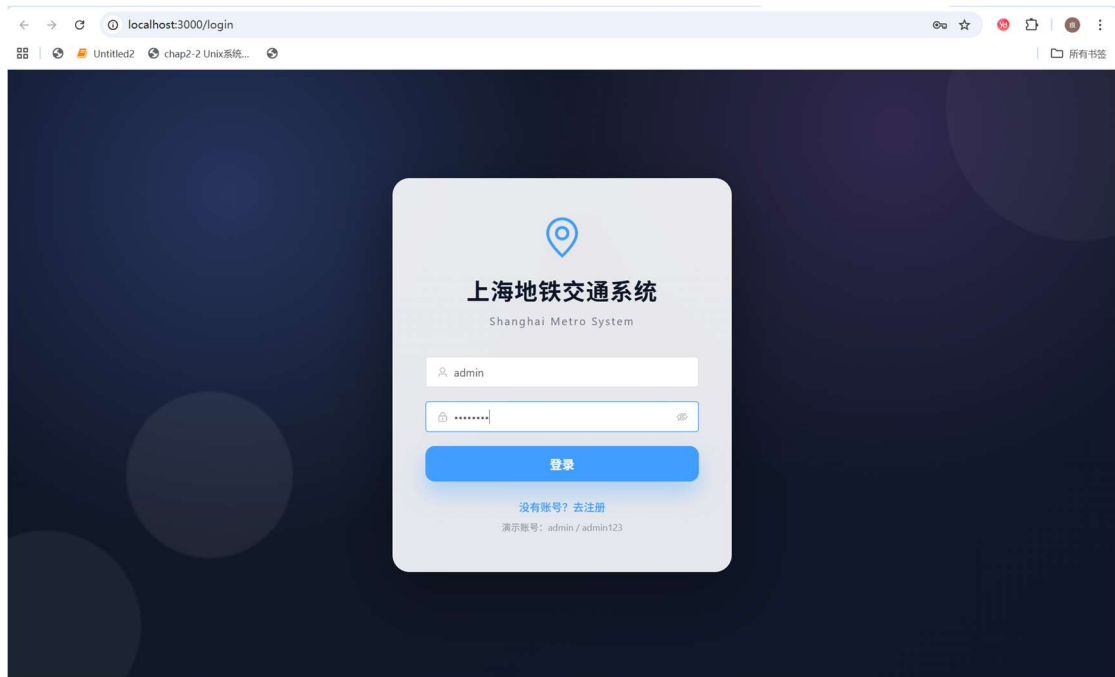
这种设计使用户无需每次访问都重新登录，同时路由守卫确保未登录用户无法直接访问受保护页面，保障系统安全性。管理员用户 (`role='admin'`) 可以访问仪表盘页面查看完整的客流分析数据，普通用户则主要使用路线规划和通勤推荐功能。

6.8 核心页面模块

系统包含四个核心页面，分别面向不同用户群体和使用场景。每个页面作为独立的 `Vue` 组件实现，通过调用 `API` 接口获取数据并渲染界面。

6.8.1 登录页 (Login.vue)

登录页是系统的入口，提供用户登录和注册功能。页面采用卡片式布局，包含用户名、密码输入框和登录/注册按钮。表单使用 `Element Plus` 的表单组件，内置校验规则确保用户输入合法（如用户名长度、邮箱格式等）。登录成功后，系统将用户信息存入 `localStorage` 并跳转至主页面。



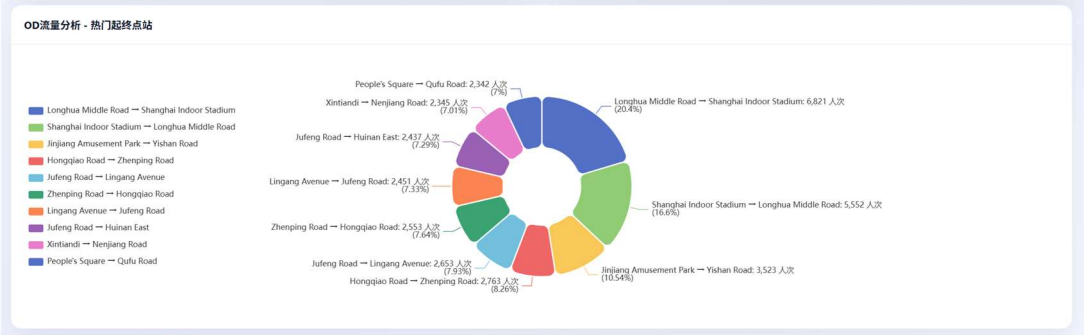
6.8.2 仪表盘（Dashboard.vue）

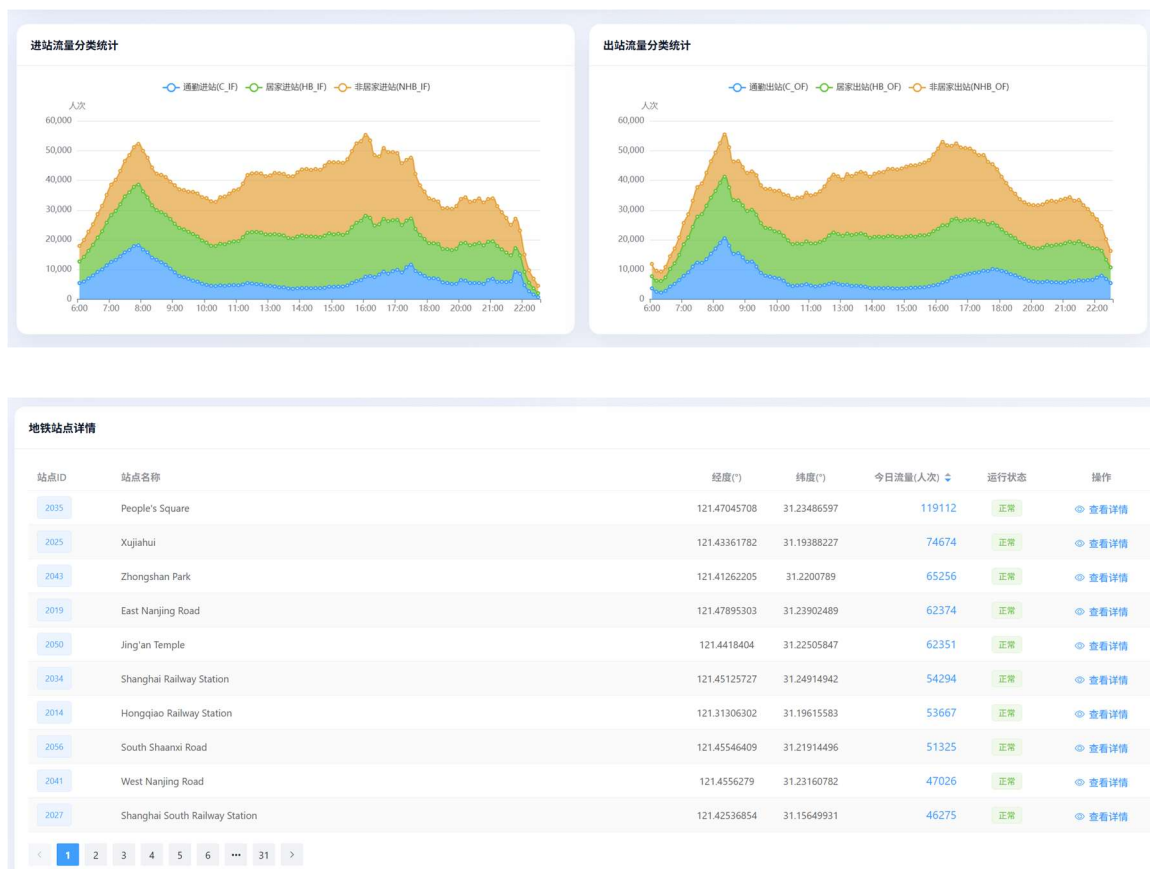
仪表盘是面向运营与规划人员的客流分析页面，是系统功能最丰富的模块。页面顶部展示统计卡片，包括站点总数、今日客流量、线路数量等关键指标；中部为多个可视化图表区域，展示实时流量趋势、站点流量排名、线路流量分布、OD 热门流量等分析结果；底部展示天气信息和进出站流量分类统计。

仪表盘主要功能模块

模块	图表类型	数据说明
统计卡片	数字展示	站点数、客流量、线路数等汇总指标
实时流量趋势	折线图	按时间段展示进出站流量变化
站点流量排名	柱状图	TOP 10 高流量站点排名
线路流量分布	饼图	各线路客流占比
OD 流量分析	桑基图	热门起终点站客流流向

模块	图表类型	数据说明
流量分类统计	堆叠柱状图	通勤/家基/非家基三类客流对比
天气信息	折线图	温度、降雨量等气象数据



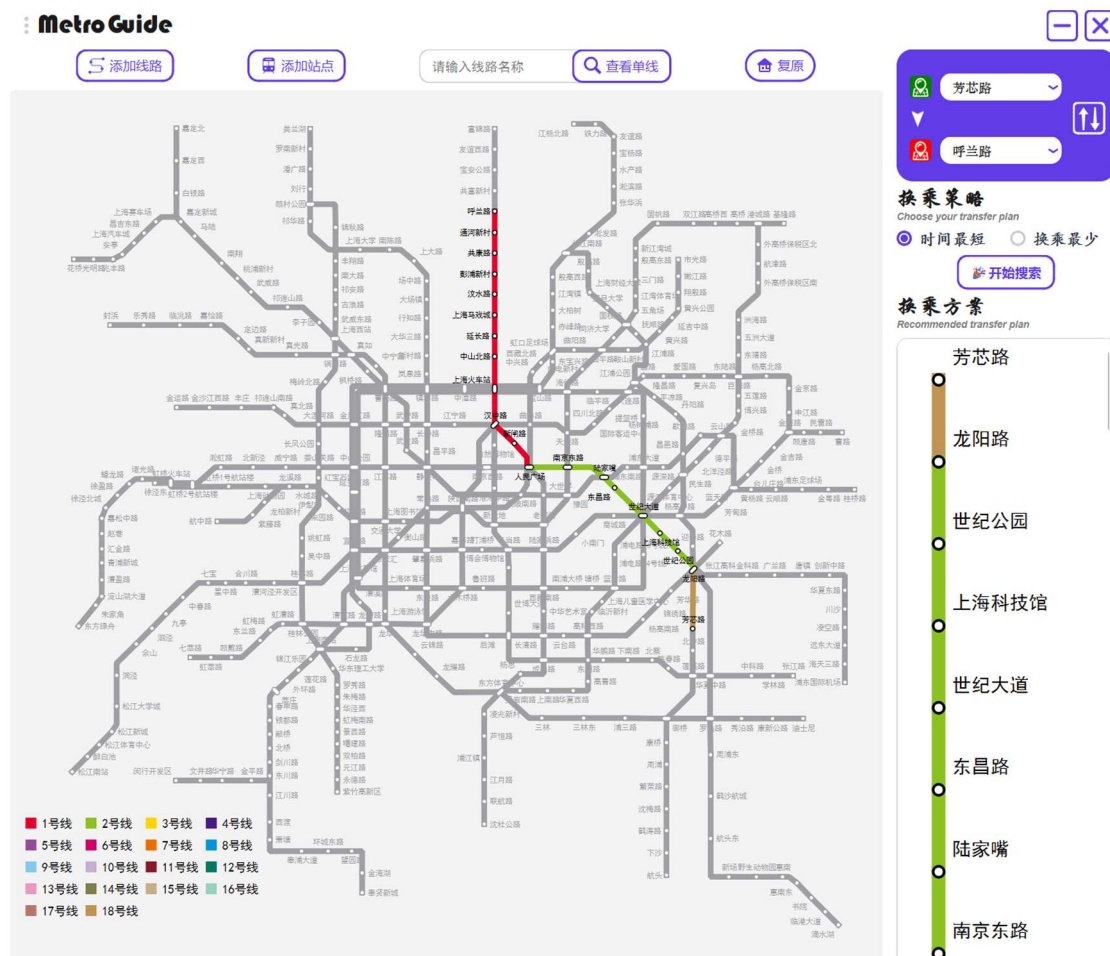


6.8.3 路线规划页（RoutePlanner.vue）

路线规划页面向公众用户，提供点对点的最优路径查询功能。页面左侧展示上海地铁线路图，右侧为路线规划面板。用户可通过下拉框选择起点和终点站（支持模糊搜索），选择换乘策略（时间最短或换乘最少），点击搜索后系统计算并展示换乘方案，包括途经站点、换乘站标记、预计用时等信息。

路线规划页主要功能有

- **站点选择：**下拉框列出所有站点，支持输入关键字模糊匹配
- **起终点交换：**一键交换起点和终点
- **换乘策略：**支持"时间最短"和"换乘最少"两种策略
- **路径展示：**以列表形式展示完整路径，换乘站特殊标记



6.8.4 通勤推荐页 (Commute.vue)

通勤推荐页专为同济大学嘉定校区师生设计,提供校区与地铁站之间的出行时刻表和推荐方案。页面展示短驳车、公交、教职工班车等多种出行方式的发车时间,并根据当前时间智能推荐最近班次。用户可查看不同方向(校区到地铁站/地铁站到校区)、不同日期类型(工作日/周末)的时刻表。





方向	时间	备注
嘉定 → 站点	05:05	学校出发
嘉定 → 站点	06:08	学校出发
嘉定 → 站点	06:48	学校出发
嘉定 → 站点	07:09	学校出发
嘉定 → 站点	07:18	学校出发
嘉定 → 站点	07:29	学校出发
嘉定 → 站点	07:38	学校出发
嘉定 → 站点	07:48	学校出发
嘉定 → 站点	07:58	学校出发
嘉定 → 站点	08:08	学校出发
嘉定 → 站点	08:18	学校出发
嘉定 → 站点	08:28	学校出发
嘉定 → 站点	08:38	学校出发
嘉定 → 站点	08:52	学校出发
嘉定 → 站点	09:12	学校出发
嘉定 → 站点	09:32	学校出发
嘉定 → 站点	09:52	学校出发
嘉定 → 站点	10:12	学校出发

6.9 前端 API 封装层

为实现前后端的高效通信，前端将所有后端 API 调用封装为独立的模块。这种设计将接口地址、请求逻辑与业务组件解耦，便于统一管理和维护。当后端接口发生变更时，只需修改对应的 API 模块，无需逐个修改调用处的代码。

6.9.1 API 模块划分

模块文件	职责说明
authApi.js	用户认证相关接口，包括注册登录、获取用户信息、更新用户信息

模块文件	职责说明
shanghaiMetroApi.js	地铁数据接口，包括站点列表、进出站流量、天气数据、OD 流量等
commuteApi.js	通勤数据接口，包括出行时刻表查询、路线元数据等

6.9.2 封装设计原则

- **统一基础路径：**所有接口共用 `API_BASE_URL` 常量，便于在开发、测试、生产环境间切换
- **异步函数封装：**使用 `async/await` 语法封装异步请求，调用方代码更简洁
- **响应数据提取：**统一处理后端返回的 `{code,message,data}` 格式，直接返回 `data` 部分供业务使用
- **错误降级处理：**部分接口在请求失败时提供降级逻辑，保证页面基本可用

6.9.3 地铁数据接口示例

```
// src/api/shanghaiMetroApi.js

const API_BASE_URL = 'http://localhost:8000/api'

/** 获取所有站点 */
export async function fetchStations() {
  const res = await fetch(`${API_BASE_URL}/metro/stations`)
  const json = await res.json()
  return json.data || []
}

/** 获取某时段的进站流量 */
export async function fetchInflowBySlot(slot) {
  const params = new URLSearchParams({ Slot: slot })
  const res = await fetch(`${API_BASE_URL}/metro/inflow?${params}`)
  const json = await res.json()
  return json.data || []
}

/** 获取热门 OD 流量 */
export async function fetchTopOD(slot, n = 10) {
```

```
const params = new URLSearchParams({ limit: n })
const res = await fetch(`${API_BASE_URL}/metro/top-od?${params}`)
const json = await res.json()
return json.data || []
}
```

6.9.4 接口调用方式

在 Vue 组件中，通过导入 API 模块并在生命周期钩子或事件处理函数中调用。

```
import { fetchStations, fetchInflowBySlot } from '@api/shanghaiMetroApi'

// 组件挂载时加载数据
onMounted(async () => {
  const stations = await fetchStations()
  const inflowData = await fetchInflowBySlot(currentSlot)
})
```

这种封装模式使业务组件专注于界面逻辑，数据获取细节由 API 层统一处理，提升了代码的可读性和可维护性。同时，API 模块可方便地添加请求拦截、错误处理、loading 状态管理等公共逻辑。

6.10 数据可视化实现

数据可视化是本系统的核心功能之一，通过图表直观展示客流数据的时空分布规律。系统使用 ECharts 作为可视化引擎，并封装为 Vue 组件便于复用。

ECharts 组件封装：为避免在每个页面重复编写图表初始化逻辑，系统将 ECharts 封装为通用组件 ECharts.vue。该组件接收图表配置对象作为属性，自动处理图表的创建、更新、销毁和窗口自适应。

```
<!-- src/components/ECharts.vue -->
<template>
  <div ref="chartRef" :style="{ width: '100%', height: height }"></div>
</template>

<script setup>
import { ref, onMounted, onUnmounted, watch } from 'vue'
import * as echarts from 'echarts'
```

```
const props = defineProps({
  option: { type: Object, required: true },
  height: { type: String, default: '400px' }
})

const chartRef = ref(null)
let chartInstance = null

onMounted(() => {
  chartInstance = echarts.init(chartRef.value)
  chartInstance.setOption(props.option)
  window.addEventListener('resize', () => chartInstance?.resize())
})

watch(() => props.option, (newOption) => {
  chartInstance?.setOption(newOption, true)
}, { deep: true })
</script>
```

使用时只需传入配置对象即可渲染图表

```
<ECharts :option="lineChartOption" height="300px" />
```

图表类型与应用场景：系统根据不同的数据分析需求，选用了多种图表类型。

图表类型	应用场景	数据来源
折线图	实时流量趋势，展示客流随时间变化	Inflow/Outflow 按时段聚合
柱状图	站点流量 TOP 10 排名	Inflow 按站点汇总排序
饼图	线路流量分布，展示各线路客流占比	按线路汇总客流
桑基图	OD 流量分析，展示热门起终点流向	ODFlow 起终点对流量
堆叠柱状图	流量分类统计，对比通勤/家基/非家基客流	C_IF/HB_IF/NHB_IF 分类数据

可视化效果说明

- **实时流量趋势图**: 横轴为时间段, 纵轴为客流量, 通过折线展示全天客流波动, 可清晰识别早晚高峰时段
- **站点流量排名图**: 横向柱状图展示流量最高的 10 个站点, 便于快速定位热门站点
- **线路流量分布图**: 饼图展示各线路客流占比, 直观反映线路负载差异
- **OD 流量桑基图**: 左侧为起点站, 右侧为终点站, 连线粗细表示流量大小, 清晰展示客流流向
- **流量分类统计图**: 堆叠柱状图对比三类客流 (通勤、家基、非家基) 的构成比例。

6.11 系统部署与运行

本节介绍系统的部署配置和启动方式, 帮助开发者快速搭建运行环境。

6.11.1 环境依赖

组件	版本要求	说明
Python	3.8+	后端运行环境
Node.js	16+	前端构建环境
MySQL	8.0+	数据库服务

6.11.2 配置数据库连接

环境变量	说明	默认值
DB_HOST	数据库主机	localhost
DB_PORT	数据库端口	3306
DB_USER	数据库用户	root
DB_PASSWORD	数据库密码	123456

环境变量	说明	默认值
DB_NAME	主库名称	jiading_commute
DATABASE_URL_METRO	分析库连接 URL	默认本地连接

第7节 总结与心得体会

本项目由小组三人协作完成，历时数月，从需求分析、数据库设计到系统开发，完整经历了一个数据库应用系统的全生命周期。在此过程中，我们不仅巩固了课堂所学的数据库理论知识，更在实践中积累了宝贵的工程经验。

通过本项目，我们深刻理解了数据库设计的规范化理论。从概念设计阶段的 E-R 图绘制，到逻辑设计阶段的关系模式转换，再到物理设计阶段的索引优化，每一步都需要综合考虑数据完整性、查询效率和存储开销。特别是在处理上海地铁客流数据时，面对进出站流量、OD 流量等亿级规模的数据表，我们学会了如何通过合理的主键设计、索引策略来保障查询性能。

项目采用前后端分离架构，让我们有机会接触完整的 Web 开发技术栈。后端使用 FastAPI 框架，学习了 RESTful API 设计规范、ORM 映射、依赖注入等现代后端开发模式；前端使用 Vue 3 框架，掌握了组件化开发、响应式数据绑定、路由守卫等前端技术。这种全栈实践使我们对软件系统的整体架构有了更清晰的认识。

客流数据的价值在于分析和洞察。通过 ECharts 图表库，我们将枯燥的数字转化为直观的折线图、柱状图、桑基图，使客流的时空分布规律一目了然。这让我们认识到，数据库不仅是存储数据的容器，更是支撑数据分析与决策的基础设施。

三人小组的协作让我们体会到软件工程中团队配合的重要性。我们通过 Git 进行版本控制，明确分工（数据库设计、后端开发、前端开发），定期同步进度，及时解决接口对接中的问题。良好的沟通和文档记录是项目顺利推进的关键。

遇到的挑战与解决：大数据量查询优化，初期 OD 流量查询响应缓慢，通过添加复合索引和优化 SQL 语句（先筛选时间段再 JOIN）显著提升性能；跨域问题，前后端分离开发时遇到浏览器跨域限制，通过配置 CORS 中间件解决；数据一致性，双数据库架构下需要注意事务边界，避免数据不一致。

第8节 未来展望

本系统目前实现了客流查询、可视化分析、路径规划等核心功能，未来可在以下方向继续完善：如引入机器学习模型，实现客流预测功能；或接入实时数据源，提供动态客流监控；又或开发移动端应用，提升用户使用便捷性。

最后，感谢张老师在项目过程中的悉心指导，感谢小组成员的辛勤付出与默契配合。本项目的完成是团队共同努力的成果，也是我们数据库课程学习的一份满意答卷。